

# Lezione: Sincronizzazione



# Obiettivi

---

- Presentare il concetto di sincronizzazione di processi
- Introdurre il problema della sezione critica
- Soluzioni software e hardware al problema della sezione critica
- Problemi classici di sincronizzazione
- Strumenti per risolvere il problema della sincronizzazione

# Background

---

- Processi possono essere eseguiti concorrentemente
  - Interrotti in ogni momento con esecuzione incomplete
- Accessi concorrenti a dati condivisi possono portare ad inconsistenze
- La consistenza richiede meccanismi per assicurare l'esecuzione ordinata di processi cooperanti

# Background

---

- Illustrazione del problema:
  - produttore consumatore con memoria limitata
  - Soluzione vista permetteva occupazione BUFFER\_SIZE - 1

# Bounded-Buffer – Produttore-Consumatore

---

- Buffer condiviso tra processi in memoria condivisa

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Buffer circolare con indici `in` (prossima free) e `out` (prima posizione full)
- Soluzione corretta, ma limite su `BUFFER_SIZE-1`
  - Buffer pieno `((in + 1) % BUFFER_SIZE) == out`
  - Buffer vuoto `in == out`

# Bounded-Buffer – Produttore

---

```
item next_produced;
while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

# Bounded Buffer – Consumatore

---

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```

# Produttore-Consumatore

---

- Illustrazione del problema:
  - produttore consumatore con memoria limitata
  - Soluzione vista permetteva occupazione `BUFFER_SIZE - 1`
  - Si può introdurre un intero **counter** che conta i buffer pieni
  - Inizialmente **counter** = 0.
  - Poi incrementato dal produttore dopo che produce un elemento nel buffer e decrementato del consumatore dopo che consuma un elemento dal buffer



# Produttore

---

```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE) ;  
        /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

# Consumatore

---

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

# Corsa Critica

- ❑ Operazioni non atomiche
- ❑ Se i due processi eseguono `counter++` e `counter--` insieme
- ❑ `counter++` può essere implementato come

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- ❑ `counter--` può essere implementato come

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- ❑ Interfogliate le due esecuzioni, inizialmente “count = 5”:

|              |                                        |                 |
|--------------|----------------------------------------|-----------------|
| S0: producer | <code>register1 = counter</code>       | {register1 = 5} |
| S1: producer | <code>register1 = register1 + 1</code> | {register1 = 6} |
| S2: consumer | <code>register2 = counter</code>       | {register2 = 5} |
| S3: consumer | <code>register2 = register2 - 1</code> | {register2 = 4} |
| S4: producer | <code>counter = register1</code>       | {counter = 6}   |
| S5: consumer | <code>counter = register2</code>       | {counter = 4}   |

# Corsa Critica

---

- Il problema della corsa critica è pervasivo
- In particolare in sistemi multicore dove multithreading è molto enfatizzato
- Occorrono meccanismi di sincronizzazione e coordinazione dei processi

# Problema della Sezione Critica

---

- Considera un sistema di  $n$  processi  $\{p_0, p_1, \dots, p_{n-1}\}$
- Ogni processo ha un segmento di **sezione critica** nel codice
  - Processi potrebbero cambiare variabili comuni, aggiornare tabelle condivise, scrivere file, etc.
  - Quando un processo entra in sezione critica gli altri processi non dovrebbero andare nella loro sezione critica
- **Problema della sezione critica** richiede un protocollo di interazione per risolverlo
  - Ogni processo deve chiedere il permesso per entrare nella sezione critica.
  - Il codice che implemente la richiesta è la **entry section**
  - Dopo la sezione ci sarà una **exit section**
  - Il codice rimanente che segue è la **remainder section**

# Critical Section

---

- Struttura generale di un processo con sezione critica  $P_i$

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

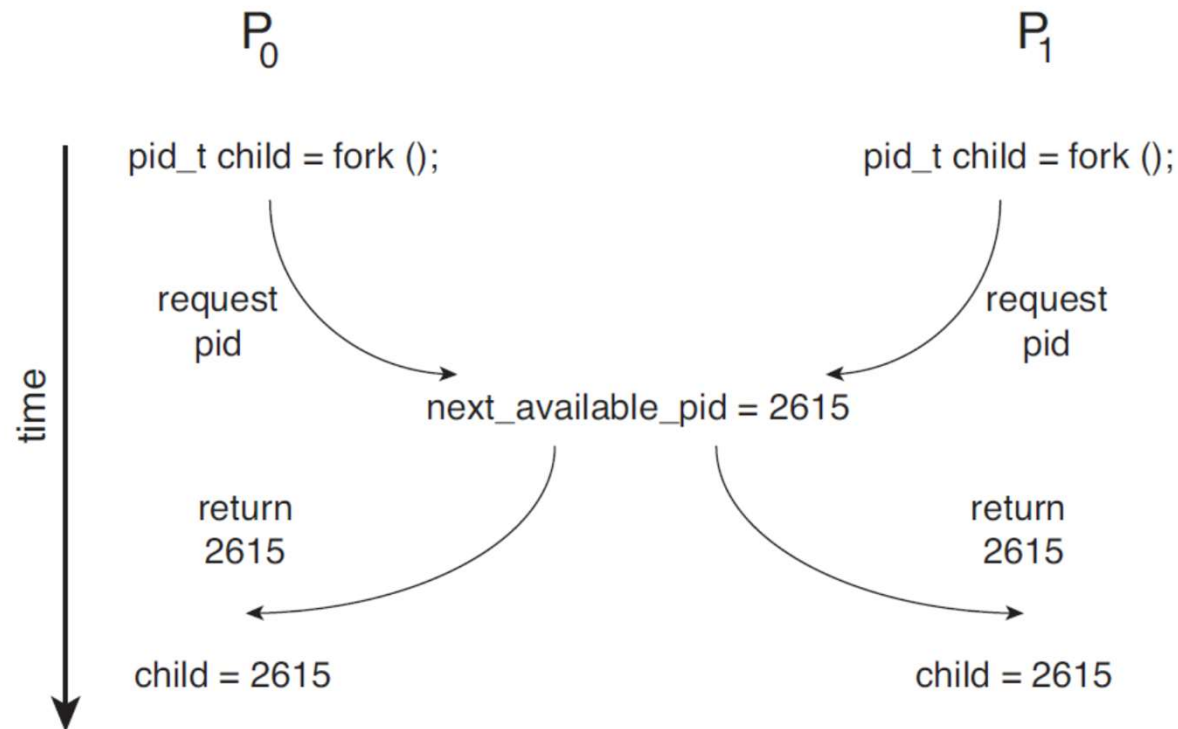
# Problema della Sezione Critica

---

1. **Mutua Esclusione** - se il processo  $P_i$  esegue la sua sezione critica nessun altro processo può eseguire la sua sezione critica
2. **Progresso** - Se nessun processo è in esecuzione nella sua sezione critica ed esistono alcuni processi che desiderano entrare nella loro sezione critica, solo i processi non in remainder section decideranno la selezione dei processi che entreranno successivamente nella sezione critica, che non può essere posticipata indefinitamente
3. **Bounded Waiting** - Deve esistere un limite al numero di volte in cui altri processi possono entrare nelle loro sezioni critiche dopo che un processo ha fatto una richiesta di entrare nella sua sezione critica prima che tale richiesta sia concessa
  - Assunto che ogni processo esegue a velocità non zero
  - Senza assunzioni sulla **velocità relativa** degli  $n$  processi

# Corse Critiche nel Kernel

- ❑ I processi  $P_0$  e  $P_1$  creano processi figlio utilizzando `fork()`
- ❑ Corse critiche su tabella di file aperti nel sistema
- ❑ Corse critiche sulla variabile kernel che rappresenta il prossimo identificatore di processo disponibile (`pid`), `next_available_pid`
  - ❑ Senza mutua esclusione, lo stesso pid potrebbe essere assegnato a due processi diversi





# Gestione della Sezione Critica

---

Due approcci

- Kernel preemptive o non-preemptive
- **Preemptive** – permette la prelazione del processo quando è in kernel mode. Complesso per SMP, permettono time sharing.
- **Non-preemptive** – esegue finché in kernel mode, si blocca, o volontariamente rilasciano la CPU
  - ▶ Non si consente l'interruzione forzata di processi attivi in modalità di sistema
  - ▶ WindowsXP/2000
  - ▶ Essenzialmente senza race condition in kernel mode

# Soluzione di Peterson

---

- ❑ Buona descrizione algoritmica del problema
- ❑ Fornisce soluzione sezione critica per due processi concorrenti
- ❑ Si assume che **load** e **store** siano istruzioni atomiche in linguaggio macchina (non interrompibili)
- ❑ Due processi condividono due variabili:
  - ❑ `int turn;`
  - ❑ `Boolean flag[2]`
- ❑ La variabile **turn** indica il processo di turno per entrare nella sezione critica
- ❑ Il **flag** array è usato per indicare se un processo è pronto per entrare in critical section.
  - ❑ `flag[i] = true` significa che il processo  $P_i$  è pronto

# Algoritmo per il Processo $P_i$

```
do {
```

```
    flag[i] = true;
```

```
    turn = j;
```

```
    while (flag[j] && turn == j);
```

```
        critical section
```

```
    flag[i] = false;
```

```
        remainder section
```

```
} while (true);
```

$P_0$ : flag[0]=true

$P_1$ : flag[1]=true

$P_1$ : turn=0

$P_1$ : while(flag[0] && turn=0);

$P_0$ : turn=1

$P_0$ : while(flag[1] && turn=1);

...

**Soluzione** — Mutua esclusione garantita dal valore di **turn**;  $P_i$  entra nella sezione critica (progresso) al massimo dopo un ingresso da parte di  $P_j$  (attesa limitata): **alternanza stretta**

# Soluzione di Peterson

---

□ Si può provare che i requisiti sono soddisfatti:

1. Mutua esclusione

$P_i$  entra in sezione critica solo se:

`flag[j] = false` oppure `turn = i`

2. Requisito di progresso è soddisfatto

3. Requisito bounded-waiting è soddisfatto

# Soluzione di Peterson

---

- ❑ In sistemi multithreaded ci sono problemi
- ❑ Esempio, due thread condividono le variabili:

```
boolean flag = false;  
int x = 0;
```

- ❑ Il primo thread esegue

```
while (!flag)  
;  
print x;
```

- ❑ Il secondo esegue

```
x = 100;  
flag = true;
```

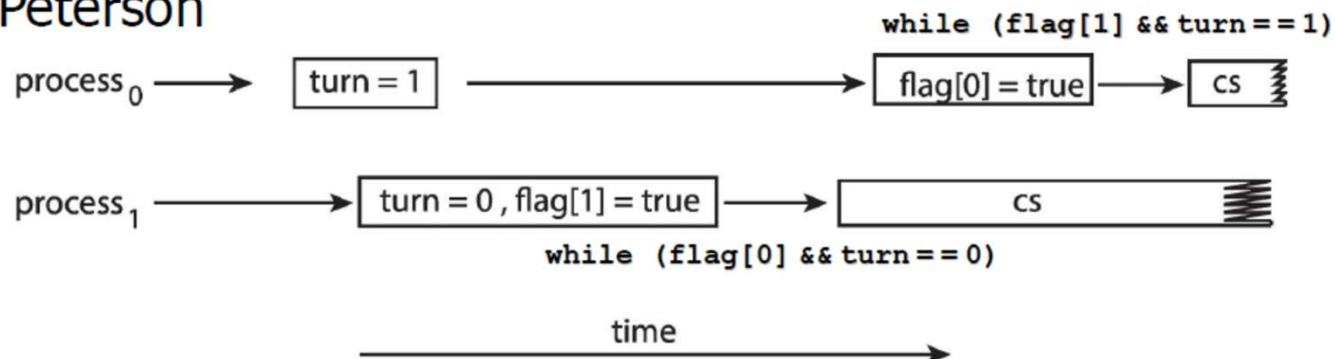
- ❑ Ci si aspetta che il primo thread stampi 100, ma non sono garantite le dipendenze tra le variabili x e flag,
- ❑ Il thread 2 potrebbe aver cambiato l'ordine delle assegnazioni e il primo thread potrebbe stampare 0

# Soluzione di Peterson

- In sistemi multithreaded si potrebbe avere accesso concorrente alla sezione

```
while(1) {  
    turn = j;  
    flag[i] = true;  
    while (flag[j] && turn == j);  
    sezione critica  
    flag[i] = false;  
    sezione non critica  
}
```

Effetto dello scambio di istruzioni nella soluzione di Peterson



# Hardware per Sincronizzazione

---

- ❑ Molti sistemi forniscono supporto hardware per implementare il codice della sezione critica
- ❑ Tutte le soluzioni che seguono si basano sul **locking**
  - ❑ Proteggere le sezioni critiche con i lock
- ❑ Uniprocessori – possono disabilitare gli interrupt
  - ❑ Il codice corrente eseguirebbe senza prelazione
  - ❑ Troppo inefficiente su sistemi multiprocessori
    - ▶ SO che lo implementano non scalabile
- ❑ Macchine moderne forniscono hardware per gestire
  - ▶ **Barriere di memoria**
  - ▶ **Istruzioni hardware**
  - ▶ **Variabili atomiche**
- ❑ Usati direttamente o utilizzati per costruire meccanismi di sincronizzazione

# Soluzioni basate su Locks

---

```
do {  
    acquire lock  
        critical section  
    release lock  
        remainder section  
} while (TRUE);
```



# Barriere di Memoria

---

- Un **modello di memoria** stabilisce quello che la memoria garantisce
- Modelli di due tipi:
  - Fortemente ordinate
    - ▶ modifica di un processore immediatamente visibile per gli altri
  - Debolmente ordinate
    - ▶ modifica non vista da tutti gli altri
- Si possono forzare cambiamenti in memoria propagati ad altri processori (barriera di memoria o recinzione di memoria)
  - Meccanismi di basso livello utilizzati da sviluppatori del kernel
  - Esempio
    - ▶ Thread 1     

```
while (!flag)
    memory_barrier();
print x;
```
    - ▶ Thread 2     

```
x = 100;
memory_barrier();
flag = true;
```

# Istruzioni atomiche

---

- ❑ Macchine moderne forniscono hardware per istruzioni atomiche
  - ❑ **Atomiche** = non-interrompibili
  - ❑ Eseguibili sequenzialmente
- ❑ Consideriamo due tipi di istruzioni
  - ❑ Test memory (word) and set (value)
  - ❑ Swap contents of two memory words

# test\_and\_set

---

Definizione:

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

1. Eseguite atomicamente
2. Restituisce il valore originale (parametro)
3. Setta il nuovo valore del parametro passato a "TRUE".

# Soluzione usando test\_and\_set()

---

- Variabile condivisa boolean lock inizializzata a FALSE
- Soluzione:

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
        /* critical section */  
    lock = false;  
        /* remainder section */  
} while (true);
```

Se lock = false, setta il lock a true ed esce dal while

Se lock = true, setta il lock a true ma non esce dal while

Per sbloccare un thread deve settare a il lock = flase

# compare\_and\_swap

---

Definizione:

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
    return temp;  
}
```

1. Eseguite atomicamente
2. Restituisce il valore originale del parametro in “value”
3. Setta la variabile “value” al valore di “new\_value” ma solo se “value” == “expected”. Cioè lo swap avviene solo con questa condizione.

Su architetture Intel x86 istruzione per CAS con lock del bus

```
lock cmpxchg <destination operand>, <source operand>
```

Comparazione tra acc e dest, se uguale dest = src, altrimenti acc = dest

# Soluzione con compare\_and\_swap

---

- Intero condiviso “lock” inizializzato a 0;
- Soluzione:

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
    /* critical section */  
    lock = 0;  
    /* remainder section */  
} while (true);
```

Non soddisfa attesa limitata  
(bounded-waiting)

Se lock = 0, restituisce 0 e setta 1, esce dal loop

Se lock = 1, restituisce 1 e rimane 1, rimane nel loop

# Bounded-waiting Mutual Exclusion con CAS

```
while (true) {  
    waiting[i] = true;  
    key = 1;  
    while (waiting[i] && key == 1)  
        key = compare_and_swap(&lock, 0, 1);  
    waiting[i] = false;  
  
    /* critical section */  
  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
  
    if (j == i)  
        lock = 0;  
    else  
        waiting[j] = false;  
  
    /* remainder section */  
}
```

waiting inizializzato a false  
lock inizializzato a 0

Il primo trova lock = 0 e setta key a 0 e lock a 1

Salta chi non attende

Se non ci sono attese sblocca lock

Se ci sono attese sblocca un processo j lasciando il lock = 1

# Bounded-waiting Mutual Exclusion con test\_and\_set

---

```
do {
    waiting[i] = true;
    key = true;
    while (waiting[i] && key)
        key = test_and_set(&lock);
    waiting[i] = false;
    /* critical section */
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = false;
    else
        waiting[j] = false;
    /* remainder section */
} while (true);
```



# Variabili atomiche

---

- ❑ CAS possono essere utilizzate per implementare altri costrutti
- ❑ Esempio incrementare una variabile

```
increment(&sequence);
```

- ❑ Implementata con CAS

```
void increment(atomic_int *v)
{
    int temp;

    do {
        temp = *v;
    }
    while (temp != compare_and_swap(v, temp, temp+1));
}
```

- ❑ Non risolvono il problema per casi più complessi

# Mutex Locks

---

- ❓ Le soluzioni precedenti sono complicate e generalmente inaccessibili ai programmatori di applicazioni
- ❓ I progettisti di SO forniscono strumenti software per risolvere i problemi della sezione critica
- ❓ Il più semplice è il mutex lock (**mutual exclusion**)
- ❓ Protegge una sezione critica prendendo **acquire()** un lock e poi rilasciandolo **release()**
  - ❓ Variabile booleana indica se il lock è disponibile o no
- ❓ Chiamate **acquire()** e **release()** atomiche
  - ❓ Solitamente implementate con istruzioni hardware atomiche
- ❓ ... ma richiede **busy waiting**
  - ❓ Il lock bloccante è chiamato **spinlock** perché il processo in attesa gira
    - ❓ Vantaggio: non fa context-switch, quindi per brevi attese è ok
    - ❓ Vantaggio: su multicore durante uno spin l'esecuzione continua
    - ❓ Uso conveniente se attesa minore di due context-switch

# acquire() e release()

---

```
❑ acquire() {
    while (!available)
        ; /* busy wait */
    available = false;;
}

❑ release() {
    available = true;
}

❑ do {
    acquire lock
    critical section
    release lock
    remainder section
} while (true);
```

Es. Implementazione di spinlock con CAS

```
void lock_spinlock(int *lock) {
    while (compare_and_swap(lock, 0, 1) != 0)
        ; /* spin */
}
```

# Semafori

- Strumenti di sincronizzazione più sofisticati (dei mutex locks) per sincronizzare processi.
- Semaforo **S** – variable intera
- Modificata con due operazioni indivisibili (atomiche)
  - **wait()** e **signal()**
    - ▶ Originariamente detti **P()** e **V()** da Dijkstra (Proberen: to test, Verhogen: to increment)
- Definizione di **wait()**

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Definizione di **signal()**  

```
signal(S) {  
    S++;  
}
```

## Operazioni Atomiche:

- Nessun altro processo può modificare il valore mentre sono in esecuzione
- Nel caso di wait sia test che decremento senza interruzioni

# Semafori Uso

- Distinzione tra due tipi di semafori:
- **Semaforo contatore** – valore intero varia su un dominio non ristretto
- **Semaforo binario** – valore intero varia tra 0 e 1
  - Come il **mutex lock**
- Controllo accesso a numero di risorse pari al valore inizializzato
- Può risolvere diversi problemi di sincronizzazione
- Esempio – vincoli di scheduling

□ Considera  $P_1$  e  $P_2$  con  $P_1$  che richiede  $S_1$  prima di  $S_2$

Crea un semaforo “**synch**” inizializzato a 0

**P1 :**

$S_1 ;$

**signal (synch) ;**

Thread join

Thread exit

Synch = 0

**P2 :**

**wait (synch) ;**

wait(S)

signal(S)

$S_2 ;$

- Può implementare un semaforo contatore **S** con un semaforo binario

# Produttori Consumatori

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0
```

Due semafori per vincoli di scheduling  
tra produttore e consumatore + un lock

```
while (true) {  
    . . .  
    /* produce an item in next_produced */  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    /* add next_produced to the buffer */  
    . . .  
    signal(mutex);  
    signal(full);  
}
```

```
while (true) {  
    wait(full);  
    wait(mutex);  
    . . .  
    /* remove an item from buffer to next_consumed */  
    . . .  
    signal(mutex);  
    signal(empty);  
    . . .  
    /* consume the item in next_consumed */  
    . . .  
}
```

# Implementazione Semaforo

---

- Garantire che due processi non eseguano `wait()` e `signal()` sullo stesso semaforo nello stesso tempo
- Problema della sezione critica dove il codice di `wait` e `signal` sono in sezione critica
- Definizioni precedenti con busy waiting

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Come evitare il busy waiting?

# Implementazione senza Busy waiting

---

- Il controllo in attesa si mette in waiting per essere svegliato da un signal (poi decide lo scheduler)
- Per implementare ogni semaforo ha una coda di attesa
  - Struttura dati semaforo:
    - ▶ valore (di tipo intero)
    - ▶ puntatore al prossimo record in lista
- ```
typedef struct{  
    int value;  
    struct process *list;  
} semaphore;
```
- Due operazioni:
  - **block** – mette il processo che chiede l'operazione nell'appropriata coda di attesa
  - **wakeup** – toglie uno dei processi in coda di attesa lo mette nella coda ready



# Implementazione senza Busy waiting

---

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
    }                                     sleep e wakeup chiamate base di Sistema  
                                         Lista di PCB  
  
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

# Implementazione Semaforo

---

- In questo caso il valore del semaforo diverge dalla definizione classica perché può essere negativo
  - Invertito ordine di decremento e attesa (definizione classica inverso)
  - Se negativo indica il numero di processi in attesa da risvegliare
- La lista dei processi in attesa si può ottenere con puntatori ai Process Control Block (PCB)
- Rimane il problema della sezione critica per il codice di **wait** e **signal**
  - Per singoli processori si può inibire l'interrupt
  - Per multicore molto inefficiente, meglio usare `compare_and_swap` o spinlocks
  - ... ma non eliminato il **busy waiting** spostato (codice wait e signal)
    - ▶ Dove il codice è breve quindi poco busy waiting se la sezione critica è raramente occupata
    - ▶ Le applicazioni potrebbero spendere molto tempo in sezioni critiche

# Deadlock e Starvation

- ❑ **Deadlock** – due o più processi sono in attesa per un evento che può essere causato solo da uno dei processi in attesa
- ❑ Siano  $S$  e  $Q$  due semafori initializzati ad 1

| $P_0$                    | $P_1$                    |
|--------------------------|--------------------------|
| <code>wait(S) ;</code>   | <code>wait(Q) ;</code>   |
| <code>wait(Q) ;</code>   | <code>wait(S) ;</code>   |
| <code>...</code>         | <code>...</code>         |
| <code>signal(S) ;</code> | <code>signal(Q) ;</code> |
| <code>signal(Q) ;</code> | <code>signal(S) ;</code> |

- ❑ **Starvation – blocco indefinito**
  - ❑ Un processo potrebbe non essere più rimosso dalla coda del semaforo su cui è sospeso
- ❑ **Priority Inversion**
  - ❑ problema di scheduling quando un processo a bassa priorità tiene un lock necessario ad un processo a più alta priorità
  - ❑ Risolto con un protocollo di **priority-inheritance**

# Problemi con i Semafori

- Uso scorretto delle operazioni dei semafori da parte dei programmatori:

- Inversione di signal e wait

- ▶ signal (mutex) .... wait (mutex)

```
signal(mutex);
```

- ▶ Violazione della mutua esclusione

```
...  
critical section
```

- Scambio di signal con wait

```
...  
wait(mutex);
```

- ▶ wait (mutex) ... wait (mutex)

```
wait(mutex);
```

- ▶ Blocco permanente

```
...  
critical section
```

```
...  
wait(mutex);
```

- Omissione di wait (mutex) o signal (mutex) (o entrambi)

- Sono quindi possibili deadlock e starvation

- Si introducono costrutti di sincronizzazione di alto livello per affrontare questi problemi

# Problemi con i Semafori

---

- Semantica dei semafori non intuitiva
- Costrutti complessi molto dipendenti dal contest e dalle inizializzazioni
- Facilmente portano ad errori